기획 워크벤치

사용 설명서

4편 — 스탯·전투 밸런스

기능별 분책 · 전체 10편 · 공통 내용은 1편

https://junhy96.github.io/gamedesign_workbench/

문서 기준: 2026년 10월 · 화면은 각 도구의 기본 템플릿 상태

차례

이 편을 읽기 전에 3
이 편의 구성 3
1편에 있는 공통 내용 (이 편에는 없음) 3
전체 편 목록 3
4. 스탯·전투 밸런스 5
4.1 무엇을 하는 도구인가 5
4.2 화면 구성 6
4.3 빠르게 시작하기 7
4.4 전투 방식 7
4.5 규칙 수식 9
4.6 엔티티와 스탯 11
4.7 아이템과 빌드 13
4.8 레벨 곡선과 승률 행렬 14
4.9 1:1 상세 16
4.10 배틀 시뮬레이터 (FIGHT · M.C) 18
4.11 레벨별 승률 (구간 리포트) 24
4.12 CP와 실제 승률 25
4.13 밸런스 도구 26
4.14 가져오기와 내보내기 28
부록 — 알고리즘·수학 사전 29
A.4 스탯·전투 밸런스 (4장) 29

이 편을 읽기 전에

이 문서는 「기획 워크벤치 사용 설명서」를 기능별로 나눈 열 편 가운데 4편, 「스탯·전투 밸런스」 편입니다. 이 편만 펼쳐도 도구를 쓰고 계산 원리를 확인할 수 있도록 본문 4장과 그 도구의 부록을 함께 담았습니다. 모든 도구에 똑같이 적용되는 내용은 되풀이하지 않고 1편의 절 번호로 가리킵니다.

이 편의 구성

- **4장 스탯·전투 밸런스 — 원본 설명서의 4장 전체**
- **부록 A.4 — 이 도구가 쓰는 계산 방법 (강화 예제는 1편 부록 A.1의 공통 예제와 같음)**

1편에 있는 공통 내용 (이 편에는 없음)

- **실행과 데이터, 헤더·도구 머리·공통 버튼, 스냅샷과 프로젝트 파일 — 1편 1.3~1.6절**
- **공식 계산과 시뮬레이션의 기본 원칙 — 1편 1.7절**
- **용어 툴팁 (i), 시행 횟수·랜덤 시드, 입력 검증, 결과 분포(P10·P50·P90·P99), 프리셋 저장·A/B 비교, 기획서 영역 — 1편 1.8절**
- **몬테카를로 시뮬레이션, 기대값·중앙값·분위수, 동적 계획법, 이분 탐색·공통 난수 — 1편 부록 A.1**
- **팀과 공유하기, 자주 묻는 질문 — 1편 11.5~11.6절**

전체 편 목록

- **1편 시작하기와 공통 기능**
- **2편 강화 확률·비용 설계**
- **3편 가챠·수집 설계**
- **4편 스탯·전투 밸런스**
- **5편 데이터 테이블**
- **6편 전투 시뮬레이터**

- **7편 패턴 프레임 검증기**
- **8편 성장 표 템플릿 (엑셀)**
- **9편 브리핑**
- **10편 부가 도구 — 다이어그램과 PDF**

4. 스탯·전투 밸런스

엔티티(캐릭터·몬스터) × 스탯 × 규칙 수식으로 레벨별 전투력(CP)과 1:1 승률을 계산하는 도구입니다. "CP는 높은데 실제로는 지는" 조합을 찾고, 상성을 원하는 승률로 맞추는 최소 변경을 계산하며, 결과를 Unity·Unreal로 내보냅니다.

4.1 무엇을 하는 도구인가

대부분의 게임은 스탯을 가중합한 **CP(Combat Power, 전투력)** 숫자를 화면에 보여 줍니다. 그런데 CP 수식이 공격 속도나 방어 감쇠 같은 상성을 제대로 반영하지 못하면, CP가 높은 캐릭터가 실제 1:1 에서는 지는 일이 생깁니다. 이 도구는 실제 전투 규칙으로 1:1 승률을 계산해 CP 순서와 비교합니다. CP 는 승패 계산에 쓰이지 않고, **CP 순서가 실제 승률 순서와 맞는지 비교하는 데만** 씁니다.

이런 질문에 답합니다

- CP가 높은데 실제로는 지는 캐릭터가 있나? CP와 실제 강함의 순위 상관은?
- 레벨 구간(Lv1~10, 11~20...)마다 누가 메타를 지배하나? 상성 순환(가위바위보)이 있나?
- 전사 대 궁수를 50%로 맞추려면 어느 스탯을 몇 배 바꾸면 되나?
- 흡혈·가시·광폭화 같은 특성이나 아이템이 승률을 몇 %p 바꾸나?
- 공격 속도 4배인 캐릭터는 턴 교대 방식과 시간 기반 방식에서 승률이 얼마나 다른가?

활용 예

- **신규 직업 추가** — 기존 직업들과의 승률 행렬을 보고, 지배하거나 완전히 밀리는 상대가 없도록 스탯을 맞춥니다.
- **CP 수식 검토** — CP 순위와 실제 순위의 상관이 낮으면 CP 수식(가중치)을 고칩니다.
- **아이템·특성 밸런스** — 아이템 하나를 껐다 켰을 때의 점수 변화로 아이템의 실제 가치를 봅니다.
- **Kalivra 사용자** — Kalivra 프로젝트(.kal)를 그대로 불러와 같은 규칙의 결과를 공식 계산으로 확인합니다.

> **팁 스탯·전투와 전투 시뮬레이터(6장) 중 무엇을 쓰나**
>
> **스탯·전투**는 사용자 정의 스탯과 규칙 수식으로 1:1 승률을 공식으로 정확히 풀고(특성·데미지 변동이 있으면 시뮬레이션), 실제 전투 흐름은 배틀 시뮬레이터 카드(4.10절)로 보여 주며, 레벨 곡선·CP 순위·최소 변경 역산까지 해 줍니다. 스탯 설계와 CP 수식 검토에 맞습니다. **전투 시뮬레이터**는 데이터 테이블의 스킬·효과·상성·시너지로 1:1과 파티 전투를 시뮬레이션합니다. 스킬 구성, 버프·디버프, 보스 패턴, 파티 조합을 볼 때 씁니다. 두 탭은 데이터를 공유하지 않습니다.

4.2 화면 구성

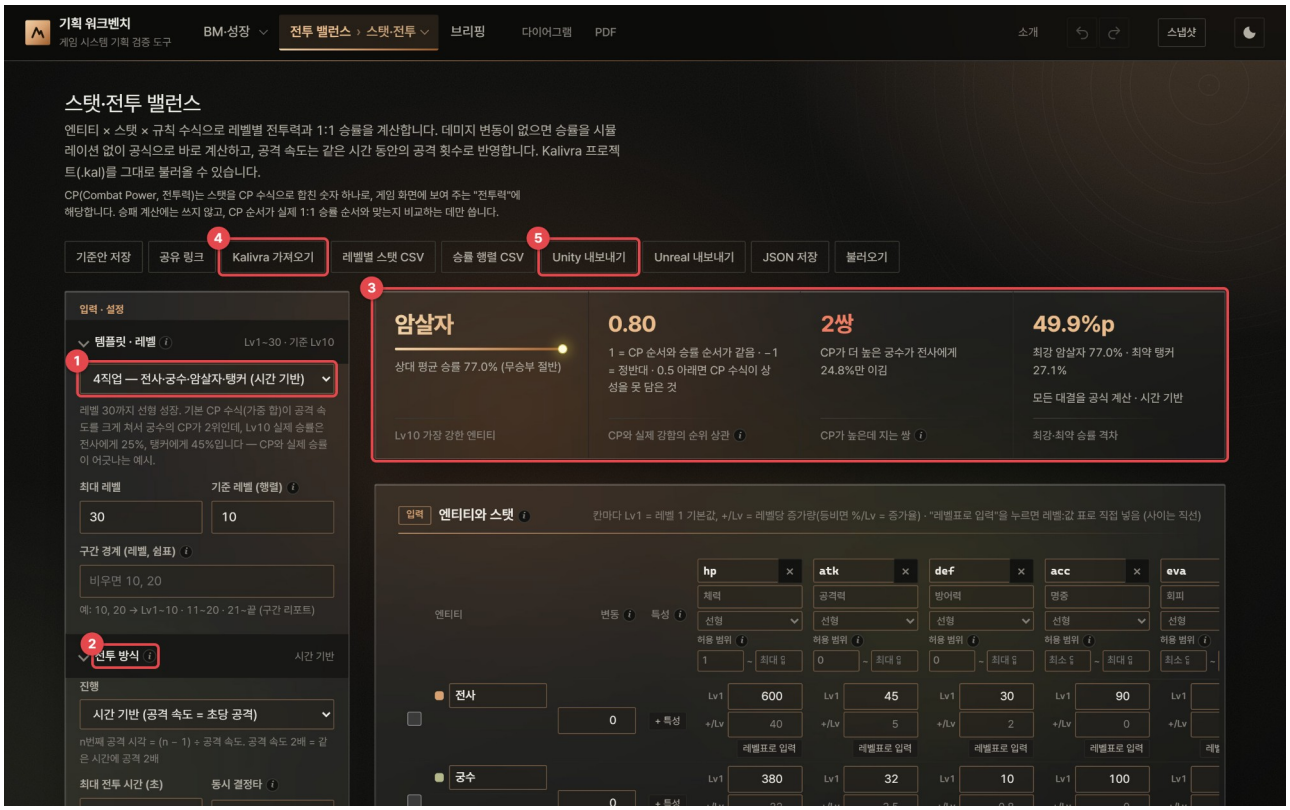


그림 29. 스탯·전투 밸런스 — 처음 화면

1	**템플릿 · 레벨** — 4직업(시간 기반), Kalivra 기본 샘플(턴 교대), 특성 예시, 빈 설계. 아래에서 최대 레벨, 기준 레벨(승률 행렬을 볼 레벨), 구간 경계(구간 리포트용, 예: 10, 20)를 정합니다.
2	**전투 방식** — 진행 방식(시간 기반 / 턴 교대), 최대 전투 시간, 동시 결정타 처리, 명중 하한·상한, 데미지 반올림, 시뮬레이션 횟수. 그 아래 **규칙 수식**이 있습니다.

3	**핵심 지표** — 기준 레벨에서 가장 강한 엔티티(상대 평균 승률), CP와 실제 강함의 순위 상관, CP가 높은데 지는 쌍 수, 최강·최약 승률 격차(아래 줄에 계산 방식: 공식 계산 / 시뮬레이션, 시간 기반 / 턴 교대).
4	**Kalivra 가져오기** — Kalivra v1.0.1 프로젝트 파일(.kal/JSON)을 이 탭 설정으로 바꿉니다. 바뀌거나 빠진 항목은 알림으로 보여 줍니다.
5	**Unity / Unreal 내보내기** — 엔진에 바로 넣을 데이터와 규칙 코드를 ZIP으로 받습니다 (4.14절).

머리의 설명 아래에 CP가 무엇인지 한 줄 설명이 있고, KPI·레벨 곡선·검증 카드에도 같은 뜻이 반복되어 처음 보는 사람도 CP를 승률과 혼동하지 않게 했습니다.

4.3 빠르게 시작하기

1. **템플릿**을 고릅니다. 4직업 템플릿은 CP 2위 궁수가 전사에게 24.8%밖에 못 이기는 "CP와 승률이 어긋나는" 예시입니다.
2. **전투 방식**과 **규칙 수식**을 우리 게임 규칙에 맞춥니다 (데미지·명중·치명·공격 속도·체력·CP 수식).
3. **엔티티와 스탯** 표에 캐릭터와 스탯(기본값·레벨당 증가)을 넣습니다.
4. **승률 행렬**에서 칸을 눌러 **1:1 상세**를 보고, **CP와 실제 승률**에서 어긋나는 쌍을 확인합니다. 실제로 싸우는 흐름은 그 아래 배틀 시뮬레이터(4.10절)의 FIGHT·M.C로 봅니다.
5. **밸런스 도구**에서 대상·상대·목표 점수를 정하면 필요한 스탯 배율이 나오고, **적용**으로 반영합니다.

4.4 전투 방식

설정	설명
시간 기반 (기본)	n번째 공격 시각 = (n − 1) ÷ 공격 속도. 공격 속도가 2배면 같은 시간에 2배 공격합니다. 두 결정타가 같은 시각이면 "동시 결정타" 설정(동전 반반 / 무승부)을 따릅니다.
턴 교대 (Kalivra 방식)	공격 속도 비율로 선공 확률을 정하고(A 선공 = A 속도 ÷ (A + B 속도)), 한 번씩 번갈아 공격합니다. 공격 속도는 선공 확률에만 쓰입니다.
최대 전투 시간 / 최대 턴	그 안에 끝나지 않으면 무승부로 따로 셉니다 (Kalivra는 양쪽 모두 패배로 셈).
명중 하한·상한	명중 수식 결과를 이 범위로 자릅니다. 치명 확률은 0~100%로 자릅니다.
데미지 반올림	내림 / 반올림 / 없음. 치명 배율까지 곱한 뒤 적용합니다.

설정	설명
시뮬레이션 횟수	데미지 변동·특성이 있을 때와 공식 계산 검증용 시뮬레이션 횟수.

두 방식의 차이는 결론을 뒤집을 만큼 큽니다. 공격 속도만 4배인 두 캐릭터(체력·공격 같음)는 턴 교대에서 80%, 시간 기반에서 100% 이깁니다. 4직업 템플릿의 전사(Lv10)는 탱커에게 시간 기반으로 84.3%, 턴 교대로 3.7%를 이깁니다. 우리 게임의 실제 전투가 어느 쪽에 가까운지 먼저 정하세요.

4.5 규칙 수식

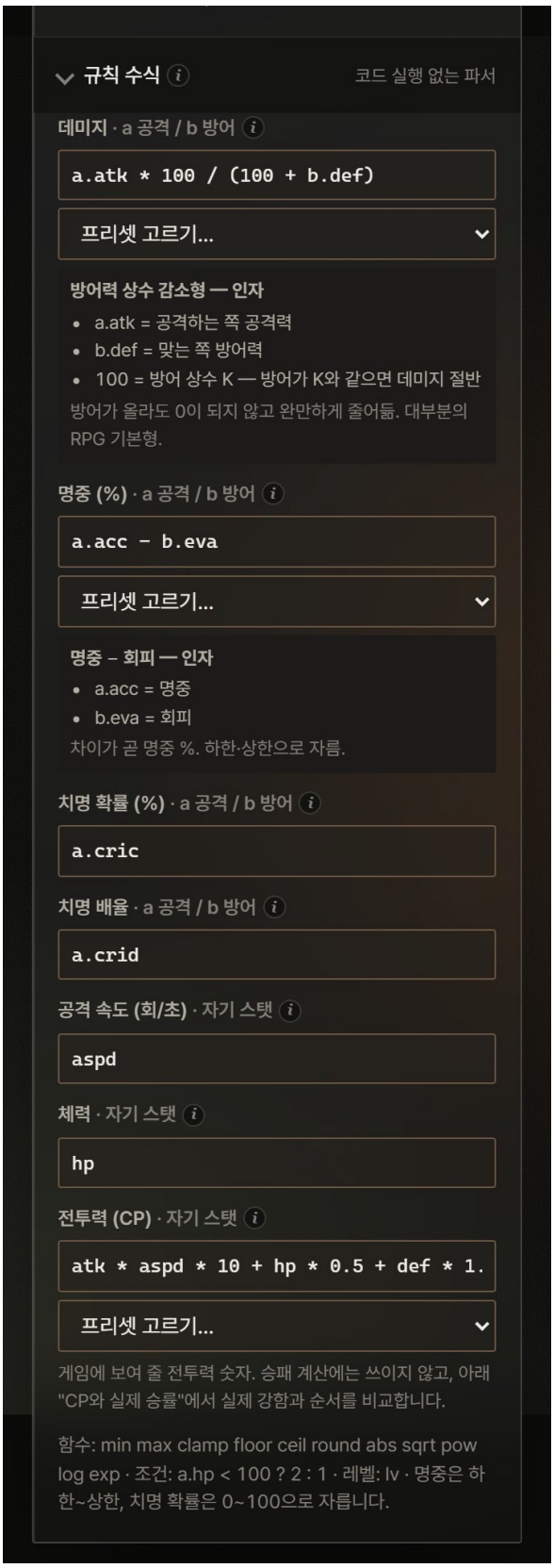


그림 30. 규칙 수식 — 데미지·명중·치명·공격 속도·체력·CP

- **데미지 · a 공격 / b 방어** — 공격하는 쪽 스탯은 `a.atk`, 맞는 쪽은 `b.def`처럼 씁니다. 프리셋: 방어 감쇠형 `a.atk * (100 / (100 + b.def))`, 차감형(최소 10%), 비율형(공 ÷ 방, 상한 3배), 레벨 차 보정.
- **명중 (%)** — 프리셋: 명중 − 회피, 비율(명중 ÷ (명중 + 회피)), 항상 명중.
- **치명 확률 (%) · 치명 배율** — a·b 스탯을 모두 쓸 수 있습니다.
- **공격 속도 (회/초) · 체력 · 전투력(CP)** — 자기 스탯만 씁니다 (`atk`, `hp`, `lv`). CP 프리셋: 가중 합 (Kalivra 기본), 기대 DPS × 체력 제곱근.
- 함수: `min max clamp floor ceil round abs sqrt pow log exp`, 조건식 `a.hp < 100 ? 2 : 1`, 레벨 `lv`. 거듭제곱은 `^` 또는 `**`.

수식에 오류가 있으면 그 칸 아래에 위치와 함께 오류가 표시되고 계산이 멈춥니다. 다른 사람이 보낸 설정 파일의 수식 칸에 코드가 들어 있어도 실행되지 않습니다.

4.6 엔티티와 스탯

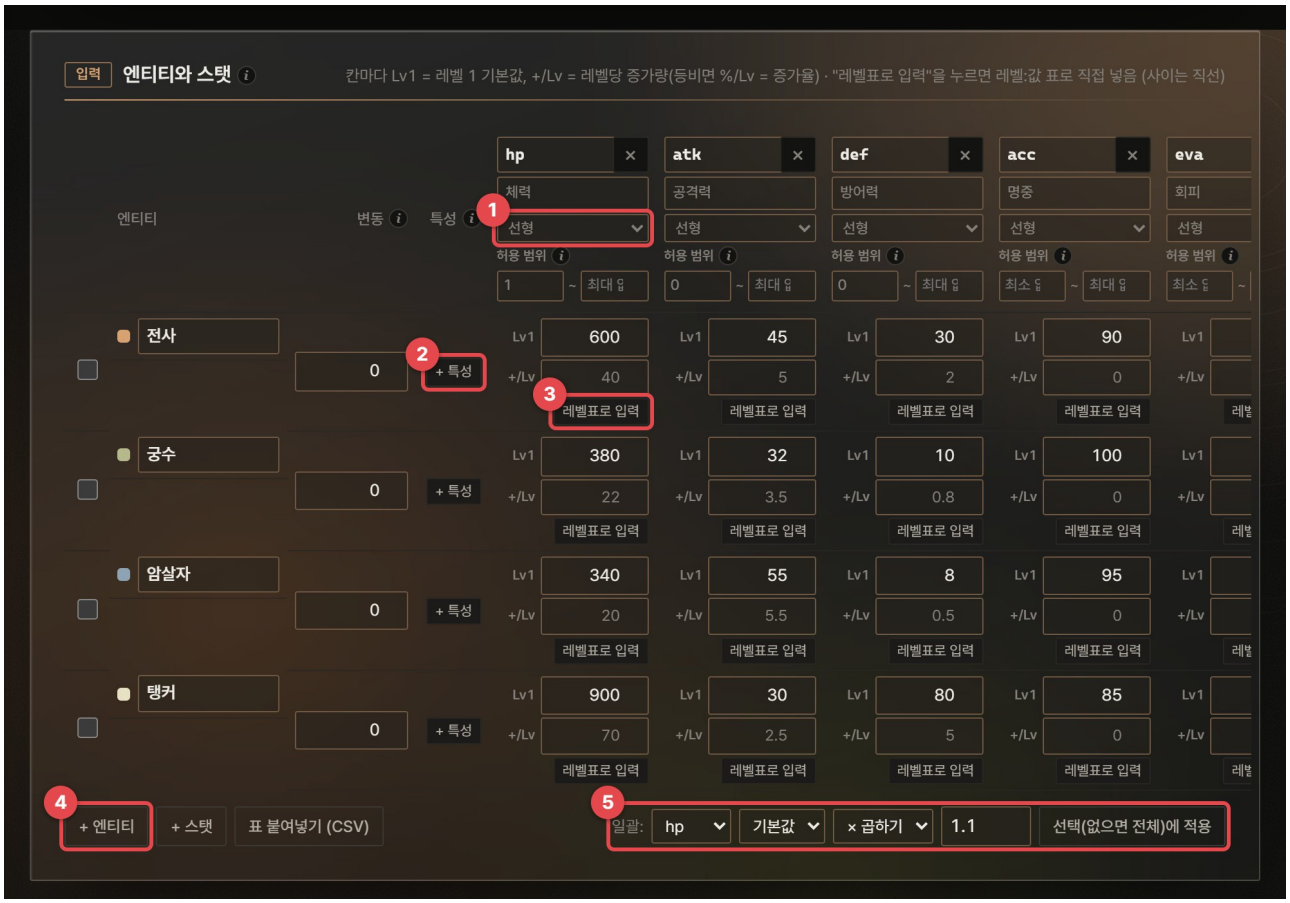


그림 31. 엔티티와 스탯 — 칸마다 위 = 기본값(Lv1), 아래 = 레벨당 증가

1	**성장 방식** — 스탯 열마다 선형(기본 + (L−1) × 증가) 또는 등비(기본 × (1 + g%)^(L−1))를 고릅니다.
2	**+ 특성** — 이 엔티티에 특성(발동 조건 + 효과)을 붙입니다. 표 아래에 특성 편집 패널이 열립니다.
3	**표** — 레벨별 값을 직접 넣습니다. 띄엄띄엄 넣으면 사이는 직선으로 잇고, 양 끝 밖은 끝값을 씁니다(구간 성장).
4	**+ 엔티티 / + 스탯 / 표 붙여넣기 (CSV)** — 행·열 추가와 엑셀 표 붙여넣기.
5	**일괄** — 고른 스탯의 기본값·성장값에 곱하기·더하기·바꾸기를 선택한 엔티티(없으면 전체)에 한 번에 적용합니다.

"변동" 열은 데미지 변동 폭(%)입니다. 변동이 0이면 공식 계산, 0보다 크면 그 엔티티가 낀 대결은 시뮬레이션으로 계산합니다.

특성 (트리거)

구성	선택지
발동 시점	전투 시작 · 매 턴(시간 기반은 매 1초) · 공격 직전 · 공격 적중 · 치명타 · 공격 빗나감 · 회피 성공 · 피격
조건	발동 확률(%), 체력 조건(자신/상대 체력 n% 이하), 최대 발동 횟수(0 = 무제한)
효과	회복 · 추가 데미지 · 이번 공격 데미지 × · 이번 공격 데미지 + · 스탯 버프
효과 기준	고정값 · 이번 공격 데미지의 % · 대상 최대 체력의 % · 대상 기본 스탯의 %
지속	스탯 버프의 지속 — 시간 기반은 초, 턴 교대는 자기 공격 수, 0 = 끝까지

프리셋으로 흡혈·가시·광폭화·선제 방어·분노·처형·재생·반격을 바로 넣을 수 있습니다. 특성이 있는 대결은 두 쪽의 공격 횟수가 서로 영향을 주므로 시뮬레이션으로 계산하고, 결과에 95% 범위가 붙습니다. 효과는 발동 한 번에 한 번만 적용하고, 확률은 발동마다 한 번만 굴리며, 쓰러뜨리는 공격 뒤에는 맞은 쪽의 피격 효과가 나오지 않습니다(이 세 가지는 Kalivra와 다르게 고친 규칙).

4.7 아이템과 빌드



그림 32. 아이템 (스탯 수정자)

1	**고정 +** — 스탯에 더하는 값.
2	**% +** — 퍼센트 증가. 같은 스탯의 %는 합산합니다.
3	**× 배율** — 곱하는 배율. 여러 아이템이면 곱합니다.
4	**+ 아이템** — 아이템 추가. 아이템마다 켜기, 대상(전체 또는 특정 엔티티), 특성(아이템에 붙는 트리거)을 정합니다.

```
최종 스탯 = (기본값 + Σ고정) × (1 + Σ% ÷ 100) × Π배율   → 그 뒤 스탯 최소·최대로 자름
(아이템 순서와 무관 — Kalivra는 순서에 따라 결과가 달랐음)
```

아이템 세트 (빌드) — "지금 아이템을 빌드로 저장"으로 지금 아이템 목록(켜기·대상·수정자·특성)에 이름을 붙여 두고, 빌드마다 엔티티 평균 승률을 표로 비교합니다. "장비 세트 A와 B 중 어느 쪽이 더 균형적인가"를 볼 때 씁니다.

4.8 레벨 곡선과 승률 행렬

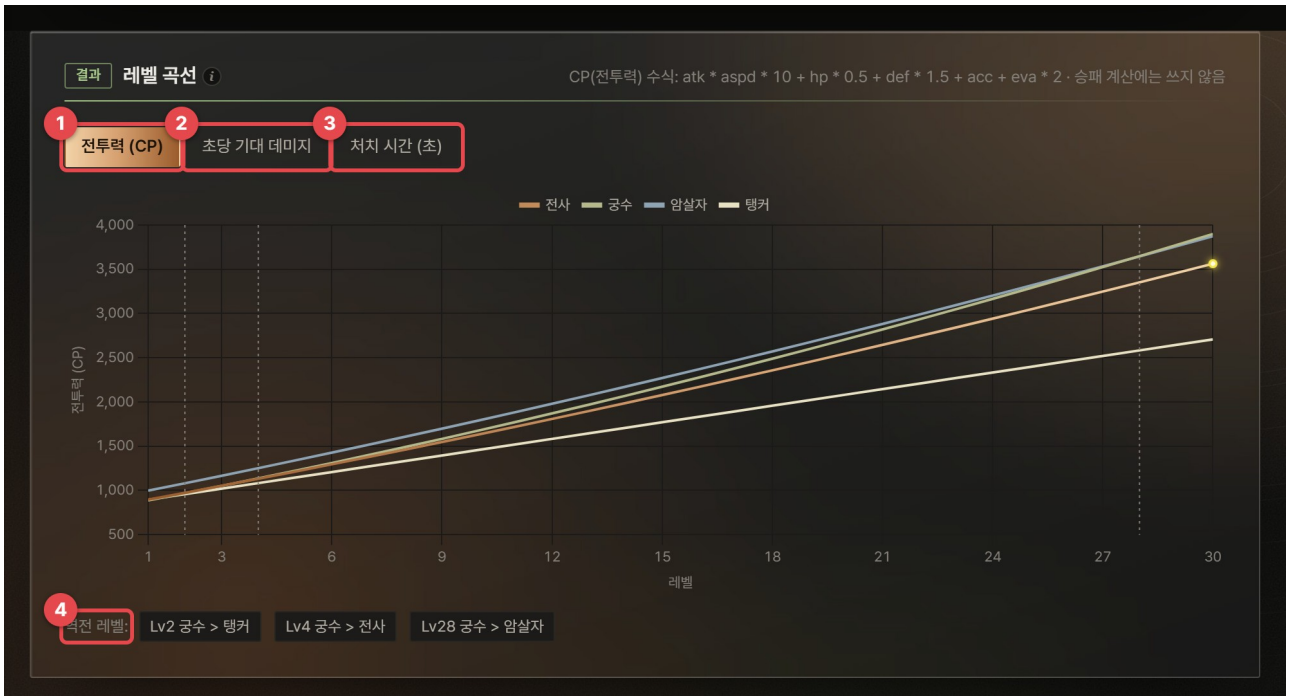


그림 33. 레벨 곡선 — 전투력(CP) / 초당 기대 데미지 / 처치 시간

1	**전투력 (CP)** — 엔티티별 레벨에 따른 CP.
2	**초당 기대 데미지** — 기준 상대(같은 레벨)에게 주는 초당 기대 데미지 (변동 제외).
3	**처치 시간** — 기준 상대를 쓰러뜨리는 데 걸리는 기대 시간.
4	**역전 레벨** — 두 곡선이 교차하는 레벨 (세로 점선). 예: Lv4부터 궁수 CP가 전사보다 높아짐.

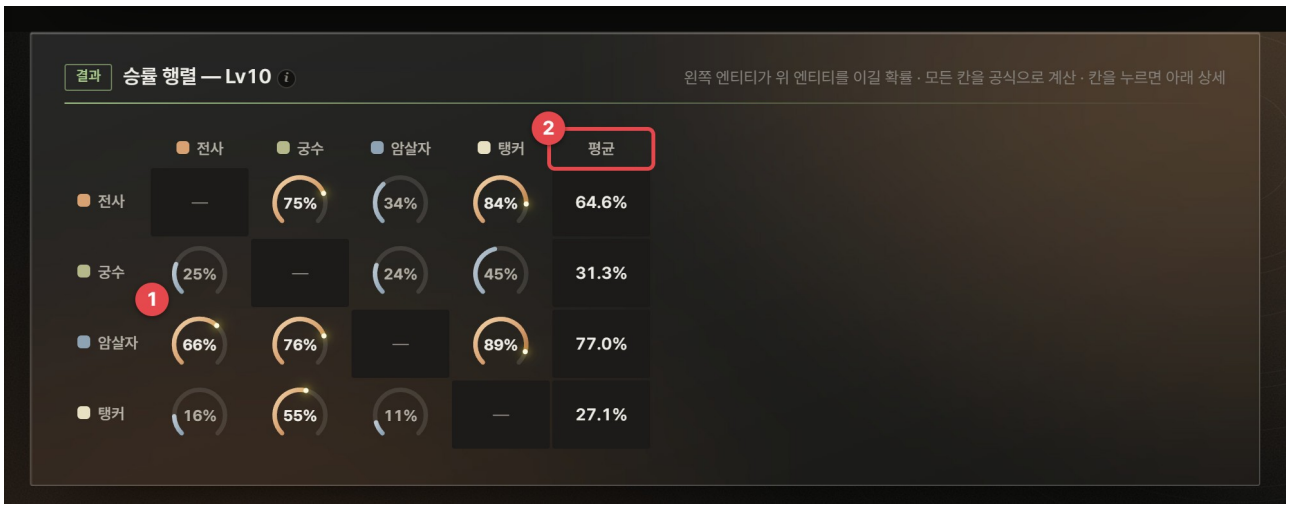


그림 34. 승률 행렬 — 왼쪽 엔티티가 위 엔티티를 이길 확률 (칸의 아치 = 승률, 탄색 = 이김 · 청회색 = 짐)

1	**칸** — 누르면 아래 1:1 상세가 그 대결로 바뀝니다. 공식 계산이 안 되는 칸(변동·특성이 있는 대결)은 시뮬레이션 값 아래에 ±오차가 함께 나오고, 95% 범위가 50%를 걸치면(어느 쪽이 강한지 확실하지 않으면) 흐리게 표시합니다. 칸에 마우스를 올리면 승·패·무와 95% 구간이 나옵니다.
2	**평균** — 그 엔티티의 모든 상대 평균 승률 (무승부는 절반 승으로 셈). 이 값으로 "실제 순위"를 정합니다.

4.9 1:1 상세



그림 35. 1:1 상세 — 승률 게이지·무·패, 공격 프로필, 공식과 시뮬레이션 대조, 수렴·전투 길이·전투 로그

1	**대결 선택** — 두 엔티티를 고릅니다 (승률 행렬 칸을 눌러도 바뀜).

2 **공식 계산 vs 시뮬레이션** — 승률·무승부·실현 명중·치명·평균 전투 길이를 나란히 비교하고, 계산값이 시뮬레이션 95% 범위 안에 있는지 표시합니다.

- **공격 프로필** — 서로에 대한 명중률, 치명 확률, 1회 데미지(일반/치명), 공격 속도, 상대 체력. 그 아래 "쓰러뜨리는 공격: 평균 10.9회(9.04초) · 중앙값 11회 · 운 나쁜 1% 17회"처럼 처치에 필요한 공격 횟수 분포가 요약됩니다.
- **시뮬레이션 수렴** — 시뮬레이션 횟수에 따른 누적 승률과 Wilson 95% 범위, 점선(공식 계산값). 범위가 ±1%p보다 넓으면 "아직 ±1%p보다 넓음"이라고 알려 줍니다.
- **전투 길이 분포** — 이긴 쪽별로 나눈 전투 길이 히스토그램(점묘, 색 = 이긴 쪽) (시간 기반은 초, 턴 교대는 턴).
- **전투 로그** — 같은 규칙의 시뮬레이션 20전투 중 하나를 골라 체력 타임라인(▲ 치명타, ◆ 특성 발동)과 사건 목록(시각, 공격자, 결과, 남은 체력)을 봅니다. "왜 이런 승률이 나오는지"를 설명할 때 유용합니다.

알고리즘 공격 횟수 분포 동적 계획법 — 1:1 승률을 시뮬레이션 없이 계산

어디에 쓰였나: 승률 행렬의 "공식 계산" 칸, 1:1 상세의 승·무·패와 처치 공격 횟수 분포, 레벨별 승률, 밸런스 도구 전부(변동·특성이 없을 때).

아이디어: 공격 한 번의 결과는 빗나감(1 − h), 일반 적중(h·(1 − c), 데미지 d), 치명(h·c, 데미지 d·m) 세 가지입니다. 상태를 (일반 적중 수 j, 치명 수 k)로 두면 "아직 살아 있음 ⇔ j·d + k·d·m < 상대 체력"이고, 공격마다 상태 확률을 다음 상태로 전진시키면 **n번째 공격에서 처음 쓰러뜨릴 확률 P(N = n)**이 모두 나옵니다.

```
시간 기반:  P(A 승) = Σₙ P(N_A = n) · P(B가 A의 n번째 공격 시각 전에 못 쓰러뜨림)
            A의 n번째 공격 시각 = (n − 1) ÷ A 공격 속도   (같은 시각이면 동전 반반 또는 무승부)
턴 교대:    P(A 승) = Σₙ P(N_A = n) · [ 선공 · S_B(n − 1) + (1 − 선공) · S_B(n) ]
            S_B(m) = P(B가 m번 공격 안에 못 쓰러뜨림),  선공 = A 속도 ÷ (A 속도 + B 속도)
```

A와 B의 처치 공격 횟수는 서로 독립(특성·버프가 없을 때)이라 이렇게 곱해서 더할 수 있습니다. 결과는 같은 설정이면 언제나 같은 값이고, Kalivra v1.0.1 엔진을 그대로 돌린 40만~140만 회 결과와 3σ 안에서 일치합니다(예: 0.72643 대 0.72676). 부록 A.4.1.

알고리즘 **Wilson 점수 구간 — 승률의 95% 범위**

어디에 쓰였나: 시뮬레이션으로 계산한 승률 칸, 1:1 상세의 시뮬레이션 열, 승률 수렴 그래프.

$$\text{중심} = (\hat{p} + z^2/2n) \div (1 + z^2/n)$$

$$\text{반폭} = z \cdot \sqrt{\hat{p}(1-\hat{p})/n + z^2/4n^2} \div (1 + z^2/n), \quad z = 1.96$$

흔히 쓰는 $\hat{p} \pm 1.96\cdot\sqrt{(\hat{p}(1-\hat{p})/n}$은 승률이 0%·100%에 가까우면 범위가 0~100%를 벗어나거나 지나치게 좁아집니다. Wilson 구간은 그런 경우에도 범위가 0~1 안에 있고 실제 포함 확률이 95%에 가깝습니다. 부록 A.4.2.

4.10 배틀 시뮬레이터 (FIGHT · M.C)

1:1 상세(4.9절)가 "이 대결의 승률은 정확히 몇 %인가"에 답한다면, 배틀 시뮬레이터는 "실제로 싸우면 어떻게 흘러가나"를 보여 줍니다. 스탯·전투 탭에서 1:1 상세 바로 아래에 있는 카드로, 이 탭의 엔티티·아이템·특성과 전투 방식·규칙 수식을 그대로 써서 전투를 한 판씩 돌립니다. 공격 쪽 하나를 골라 상대 한 명 또는 전체와 정해진 횟수만큼 싸우게 하고(FIGHT), 한 쌍을 1만 번 돌려 선공·남은 체력·오버킬까지 봅니다(M.C). 난수는 시드를 고정해 같은 설정이면 같은 숫자가 나옵니다. 화면 구성은 Kalivra의 Battle Simulator(FIGHT · M.C 버튼)를 따랐습니다.

팁 **이름이 비슷한 6장 전투 시뮬레이터와 다른 점**

이 카드는 스탯·전투 탭의 1:1 대결만 다루고 스킬·버프·파티가 없습니다. 데이터 테이블(5장)의 스킬·효과·시너지로 파티 전투를 돌리는 것은 별도 탭인 전투 시뮬레이터(6장)입니다. 두 도구는 데이터를 공유하지 않습니다.

이런 질문에 답합니다

- 전사는 Lv10 궁수에게 몇 % 이기고, 보통 몇 초 만에 끝나나? 이기는 판과 지는 판은 어떻게 흘러가나?
- 턴 교대 규칙에서 선공을 잡으면 승률이 얼마나 오르나? 선공 운이 승패를 좌우하는 대결인가?

- 이길 때 체력을 얼마나 남기나 — 여유 있게 이기나, 아슬아슬하게 이기나?
- 초반·중반·후반 중 언제 강하고 언제 약한가?

FIGHT — 상대마다 정해진 횟수만큼



그림 36. 배틀 시뮬레이터 — FIGHT (4직업 템플릿, 전사 대 전체 Lv10, 상대마다 100전투)

1	**공격 쪽 · 상대** — 싸울 두 쪽. 상대에 "전체 (리그)"를 고르면 공격 쪽이 나머지 엔티티 모두와 차례로 싸웁니다.
2	**레벨 · 횟수** — 두 엔티티에 함께 쓰는 레벨(처음에는 기준 레벨)과, FIGHT에서 상대마다 돌릴 전투 수(1~5,000).
3	**M.C · FIGHT** — M.C는 상대 한 명과 10,000번, FIGHT는 상대마다 "횟수"만큼 싸웁니다. 규칙이나 스탯을 바꾸면 결과는 지워집니다.

4	**승률 · 평균 길이** — 상대마다 공격 쪽 승률(초록, 왼쪽 축)과 평균 전투 길이(청회색, 오른쪽 축 — 시간 기반은 초, 턴 교대는 턴). 막대에 마우스를 올리면 무승부 비율과 이긴 판만의 평균 길이가 나옵니다.
5	**VS 상대** — 상대별 묶음. 평균 길이와 승률(50% 이상 초록, 미만 빨강). 눌러서 접고 폅니다.
6	**전투 로그 ◀ ▶** — 앞 100전투를 하나씩 넘겨 봅니다. 결과·길이(턴 교대면 선공도)가 나오고, 아래에 시각(또는 턴)마다 공격·빗나감·치명타·특성 발동과 두 쪽의 남은 체력이 이어집니다.
7	**스탯** — 그 레벨의 최종 스탯(아이템 포함)을 엔티티별 카드로 보여 줍니다.

4직업 템플릿의 처음 상태(시간 기반, 기준 레벨 10)에서 전사로 전체와 100번씩 싸우면 궁수에게 75번(평균 8.71초), 암살자에게 32번(7.71초), 탱커에게 79번(36.56초) 이깁니다. 궁수 상대 첫 전투의 로그를 넘겨 보면 두 쪽 모두 0초에 첫 공격을 하고, 전사는 한 대에 76(치명타 115), 궁수는 49(치명타 83)를 넣습니다. 궁수가 공격 속도 1.68로 전사(1.09)보다 1.5배 자주 치지만, 체력 578인 궁수는 여덟 대면 쓰러지고 체력 960인 전사는 스무 대를 버텨, 첫 전투는 전사가 체력 221을 남기고 7.34초에 이깁니다. 암살자에게 지는 이유도 스탯 카드에서 보입니다 — 암살자의 회피 34.5 때문에 전사의 명중은 90 − 34.5 = 55.5%로 떨어지고, 암살자는 치명 확률 35%에 치명 배율 2배입니다.

상대를 한 명(예: 궁수)으로 고르고 FIGHT를 누르면 구간별 승률 카드가 함께 나옵니다. 구간마다 레벨별 승률을 평균한 값으로, 공식으로 풀리는 레벨은 공식 계산, 아니면 시뮬레이션이고 구간 경계는 왼쪽 "템플릿 · 레벨"을 따릅니다(4.11절의 구간 리포트와 같은 경계). 전사 대 궁수는 Lv1~10 73.5%, Lv11~20 75.9%, Lv21~30 73.8%로, 레벨과 상관없이 전사가 비슷하게 앞섭니다.

M.C — 한 쌍을 1만 번



그림 37. 배틀 시뮬레이터 — M.C (전사 대 궁수 Lv10, 10,000번)

1	**전체 승률** — 공격 쪽 승률과 Wilson 95% 범위. 공식으로 풀리는 대결이면 공식 계산값도 함께 나오고 (특성·데미지 변동이 있으면 시뮬레이션만), 무승부가 있으면 그 비율도.
2	**선공일 때 승률** — 턴 교대에서 공격 쪽이 먼저 공격한 판만의 승률, 선공 비율, 후공일 때 승률. 시간 기반은 둘 다 0초에 시작하므로 이 예처럼 "—"로 표시합니다. 두 승률이 20%p 넘게 벌어지면 빨갛게.
3	**평균 길이 · 계산 시간** — 모든 판의 평균 전투 길이와 공격 쪽이 이긴 판만의 평균, 1만 번에 걸린 시간.
4	**실현 치명타율 · 회피율** — 치명타율은 적중 중 비율(아래 작은 글씨는 빗나간 공격까지 분모에 넣은 Kalivra 표기), 회피율은 상대의 공격 중 빗나간 비율.
5	**평균 DPS(DPT) · 평균 오버킬** — 공격 쪽이 가한 데미지 합 ÷ 전투 길이 합. 시간 기반은 초당(DPS), 턴 교대는 턴당(DPT)입니다. 오버킬 = 공격 쪽이 이긴 판에서 마지막 공격이 상대 체력을 넘긴 양의 평균.
6	**전투 길이 분포** — 이긴 쪽별로 쌓은 막대 — 1:1 상세의 전투 길이 분포와 같은 형식.
7	**승리 시 남은 체력 분포** — 이긴 쪽이 전투 끝에 남긴 체력(절댓값)을 10칸으로. 0 근처에 몰리면 아슬아슬한 승리, 오른쪽이면 여유 있는 승리입니다.

4.3절에서 본 "CP 2위 궁수가 전사에게 24.8%밖에 못 이기는" 대결을 1만 번 돌리면 전사 승률은 75.13%(95% 74.3~76.0%)이고, 공식 계산 75.16%가 범위 안에 있습니다. FIGHT 100번의 75.0%와 같은 값을 훨씬 좁은 범위로 보여 주는 셈입니다. 실현 치명타율 10.1%는 전사의 치명 확률 10%, 회피율 5.0%는 궁수 명중 100 − 전사 회피 5 = 95%와 맞습니다. 작은 글씨의 공격 중 7.3%는 빗나간 공격까지 분모에 넣은 값으로, 10.1% × 전사 명중률 72%(90 − 17.7)와 같습니다.

남은 체력 분포에서 전사의 승리는 0부터 530 근처까지 퍼져 있고 중앙값이 187입니다. 최대 체력 960의 5분의 1 정도만 남기고 이기는 판이 보통이라는 뜻입니다. 반대로 궁수의 승리는 절반 이상(55%)이 첫 칸(0~58)에 몰려 있어 이겨도 거의 아슬아슬하게 이기고, 전사도 전체 판의 9.8%는 58 이하만 남기고 이깁니다. 두 쪽 모두 한두 대 차이로 갈리는 판이 많으니, 궁수의 체력이나 공격력을 조금만 바꿔도 이 판들이 뒤집혀 승률이 움직입니다. 평균 DPS 68.8은 전사가 1초에 평균 69를 넣는다는 뜻으로, 궁수 체력 578을 깎는 데 약 8.4초가 필요합니다 — 전사가 이긴 판의 평균 길이 8.25초와 맞습니다.

선공 효과는 턴 교대에서만 나타납니다. 왼쪽 전투 방식의 진행 방식을 턴 교대로 바꾸고 전사 대 암살자로 M.C를 돌리면, 전사는 45.1%의 판에서 먼저 공격하고 선공일 때 58.6%, 후공일 때 47.6%를 이겨 전체 승률이 52.6%입니다. 11%p 차이는 첫 공격 한 번이 결과를 꽤 바꾼다는 뜻입니다. 두 승률이 20%p 넘게 벌어지면 선공 운이 승패를 좌우한다는 신호이므로, 선공 확률을 정하는 공격 속도나 첫 공격의 데미지를 다시 봅니다.

알고리즘 배틀 시뮬레이터 — 선공으로 나눈 승률과 Kalivra와 다른 점

어디에 쓰였나: M.C의 선공일 때 승률, FIGHT·M.C 전체.

```
승률 = 선공 비율 × 선공일 때 승률 + (1 − 선공 비율) × 후공일 때 승률
     = 0.451 × 0.586 + 0.549 × 0.476 ≈ 0.526      (턴 교대, 전사 대 암살자 Lv10)
선공 비율 ≈ A 공격 속도 ÷ (A 공격 속도 + B 공격 속도) = 1.09 ÷ (1.09 + 1.29) ≈ 0.458
```

전체 확률의 법칙: 승률을 "선공한 판"과 "후공한 판"으로 나눠 가중 평균한 것입니다. 두 승률의 차이가 선공 운의 영향이고, 선공 비율은 턴 교대 규칙대로 공격 속도 비에서 나옵니다(4.4절). 1만 번 중 실제 선공 비율 45.1%는 이 비 45.8%와 시뮬레이션 오차 안에서 맞습니다.

Kalivra와 다른 점: ① 시드를 고정해 같은 설정이면 같은 숫자(Kalivra는 Math.random이라 누를 때마다 바뀜). ② 무승부를 따로 셈. ③ Kalivra의 "Realized Crit %"는 빗나간 공격까지 분모에 넣어 치명 확률보다 낮게 나오므로, 적중 중 비율과 나눠 보여 줌. ④ 구간별 승률을 레벨당 10전투 대신 공식 계산(또는 더 많은 시뮬레이션)으로. 부록 A.1.1(몬테카를로), A.4.2(Wilson 구간).

4.11 레벨별 승률 (구간 리포트)



그림 38. 레벨별 승률 — 엔티티별 평균 승률 곡선과 구간 표, 구간 지배 경고

모든 레벨에서 승률 행렬을 계산해 엔티티별 평균 점수(승 + 무승부 ÷ 2)를 곡선으로 그리고, "템플릿 · 레벨"의 구간 경계로 나눈 구간별 평균을 표로 보여 줍니다. 구간 평균으로 모든 상대에게 55% 이상 이기는 엔티티는 "지배"로 경고합니다. 시뮬레이션이 필요한 쌍이 많으면 전투 6만 회 예산 안에서 반복 수를 줄이거나 레벨을 건너뛰고, 화면이 멈추지 않도록 레벨마다 나눠 계산합니다.

4직업 템플릿(Lv1~30, 구간 Lv1~10 · 11~20 · 21~30)에서는 세 구간 모두 암살자가 구간 평균으로 모든 상대에게 55% 이상 이겨(평균 승률 82.4% → 76.0% → 72.5%) "지배" 경고가 세 번 나옵니다. 전사는 58.7%에서 69.7%로 후반에 강해지고, 탱커는 29.9%에서 21.3%로 약해집니다.

상성 순환: A가 B를, B가 C를, C가 A를 55% 이상 이기는 세 엔티티를 찾아 "의도한 가위바위보 구조인지 확인하세요"라고 알려 줍니다.

4.12 CP와 실제 승률

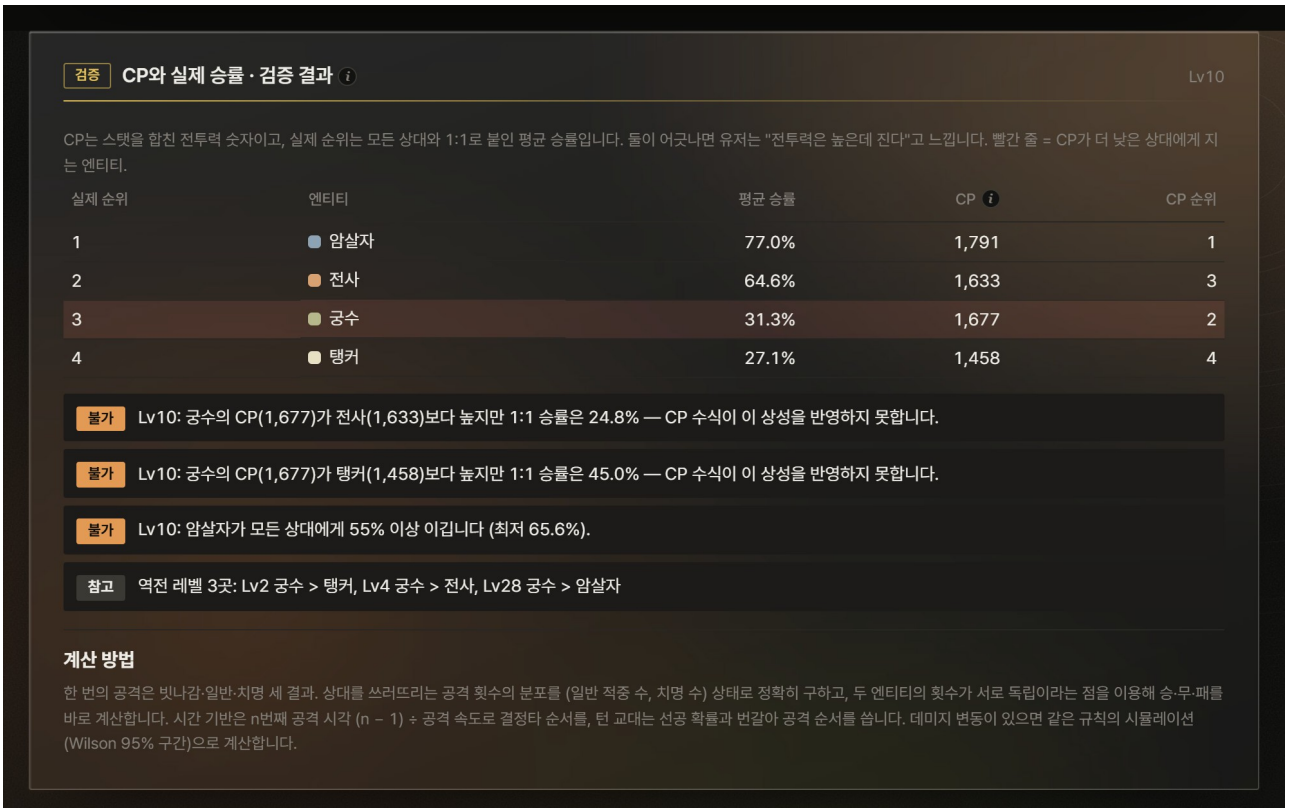


그림 39. CP와 실제 승률 · 검증 결과 — 빨간 줄 = CP가 더 낮은 상대에게 지는 엔티티

실제 순위(모든 상대와의 평균 승률)와 CP 순위를 나란히 놓고, CP가 높은데 지는 쌍을 빨간 줄과 검증 문장으로 보여 줍니다. 기본 템플릿: CP 2위 궁수(1,677)가 CP 3위 전사(1,633)에게 24.8%, CP 4위 탱커(1,458)에게 45.0%만 이깁니다. 기본 CP 수식이 공격 속도를 크게 쳐서 생긴 어긋남입니다.

> **알고리즘 스피어만 순위 상관**
>
> **어디에 쓰였나**: 핵심 지표 "CP와 실제 강함의 순위 상관", 브리핑 지표.
>
> ρ = (CP 순위, 실제 순위) 두 순위 사이의 피어슨 상관계수 — 동점은 평균 순위
>
> 동점이 없으면 $\rho = 1 - 6\,\Sigma\, d_i^2 \div \left(n(n^2 - 1) \right)$, d_i = 엔티티 i의 CP 순위 − 실제 순위
>
> 값의 크기가 아니라 **순서**가 얼마나 같은지만 봅니다. 1이면 CP 순서와 실제 순서가 완전히 같고, −1이면 정반대입니다. 기본 템플릿은 0.80 — 대체로 맞지만 한 쌍 이상이 뒤집혀 있다는 뜻입니다. 부록 A.4.3.

4.13 밸런스 도구



그림 40. 밸런스 도구 — 점수를 목표로 만드는 최소 변경과 민감도

1	**대상 · 상대 · 레벨** — 점수를 볼 엔티티와 상대(한 명 또는 모든 상대 평균), 레벨.
2	**목표 점수 (%)** — 점수 = 승 + 무승부 ÷ 2. 보통 50%. 민감도 변동폭(±5/10/20%)도 여기서 고릅니다.

3	**적용** — 그 스탯을 필요 배율만큼 바꿉니다 (기본값·성장값·표 값을 모두 같은 배율로).

- **점수를 목표로 만드는 최소 변경** — 대상의 스탯 하나만 ×k 했을 때 목표 점수에 닿는 k를, 필요한 변화가 작은 순으로 보여 줍니다. 공격 횟수가 정수라 점수는 계단 모양이므로 "목표를 처음 넘는 값"과 그때의 점수를 함께 표시합니다. 지금 점수가 목표보다 높으면 목표까지 내리는 배율(여유)이 나옵니다.
- **민감도** — 대상과 상대의 스탯을 하나씩 ±10% 바꿨을 때 대상 점수가 몇 %p 바뀌는지 토네이도로 보여 줍니다. 오른쪽 = 대상에게 유리.

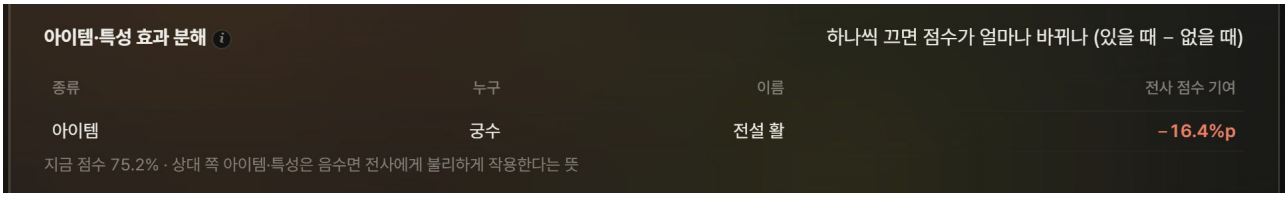


그림 41. 아이템·특성 효과 분해 — 하나씩 끄면 점수가 얼마나 바뀌나

대상에게 걸린 아이템·특성(상대가 한 명이면 상대 것도)을 하나씩 껐을 때의 점수 차이(있을 때 − 없을 때)입니다. 상대 쪽 아이템이 음수면 대상에게 불리하게 작용한다는 뜻입니다. 그림에서 궁수의 "전설 활"이 전사 점수를 16.4%p 깎습니다.

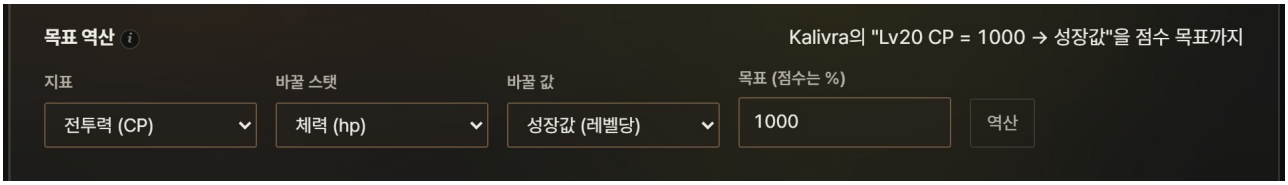


그림 42. 목표 역산 — Kalivra의 "Lv20 CP = 1000 → 성장값"을 점수 목표까지

지표(전투력 CP / 초당 기대 데미지 / 점수)와 바꿀 스탯, 바꿀 값(성장값 / 기본값 / 전체 배율), 목표를 넣고 **역산**을 누르면 목표에 맞는 값을 찾아 적용합니다. 예: "Lv20 전사 CP = 1000이 되려면 체력 성장값은?" 또는 "Lv20 전사 대 궁수 점수 50%가 되려면 공격력 성장값은?"

알고리즘 로그 이분 탐색과 같은 시드 비교

최소 변경·목표 역산: 스탯 배율 k를 키우면 점수는 단조로 변합니다. 1/64배~64배 범위에서 로그 축 이분 탐색으로 목표를 처음 넘는 k를 찾습니다(공식 계산 40회, 시뮬레이션 22회 반복). 그 범위에서 닿지 않으면 "이 스탯만으로는 닿지 않음"과 끝에서의 점수를 보여 줍니다.

시뮬레이션이 필요한 쌍: 배율마다 같은 시드(같은 난수열)로 시뮬레이션해서, 설정을 조금 바꿨을 때의 차이가 난수 잡음에 묻히지 않게 합니다(공통 난수 기법). 부록 A.1.4.

4.14 가져오기와 내보내기

버튼	내용
Kalivra 가져오기	Kalivra 프로젝트(.kal / JSON)를 읽어 엔티티·스탯·아이템·규칙·특성을 이 탭 설정으로 바꿉니다. 특성 트리거 이름을 발동 시점으로 대응시키고, 대응하지 못한 트리거·효과는 알림으로 알려 줍니다.
레벨별 스탯 CSV	엔티티 × 레벨 × 스탯 값(아이템 포함) 표.
승률 행렬 CSV	기준 레벨의 승률 행렬.
Unity 내보내기	ZIP: BattleData.json, Scripts/BattleRules.cs(규칙 수식을 C# 코드로), Scripts/BattleData.cs(ScriptableObject), Scripts/Editor/BattleDataImporter.cs, README.txt
Unreal 내보내기	ZIP: DT_BattleEntities.csv/json, DT_BattleItems.csv/json(DataTable 행 배열), Source/BattleRows.h(행 구조체), Source/BattleRules.h(규칙 수식을 C++로), README.txt

주의 엔진 내보내기는 출발점입니다

내보낸 C#·C++ 코드는 수식 파서의 나무를 그대로 옮긴 것으로, 실제 엔진 프로젝트에서 컴파일해 보지 않았습니다. 프로젝트의 네임스페이스·빌드 설정에 맞게 확인한 뒤 쓰세요. ID는 문자열로 내보내 Kalivra의 정수 넘침 문제를 피했고, 특성의 트리거·효과는 모두 내보냅니다.

부록 — 알고리즘·수학 사전

이 부록의 공통 예제 (1편 부록 A.1과 같은 예제)

아래 작은 강화를 이 편의 부록 곳곳에서 계속 씁니다. 몬테카를로·분위수·이분 탐색으로 이 예제를 푸는 과정은 1편 부록 A.1에 있습니다.

```
시도 1회 = 1,000원,  시작 +1 → 목표 +3
+1에서 시도: 성공 50% → +2,  실패 50% → +1 유지
+2에서 시도: 성공 40% → +3(목표),  실패 60% → +1로 하락
```

A.4 스탯·전투 밸런스 (4장)

스탯·전투 탭의 계산은 한 줄로 이어집니다. 먼저 "몇 번째 공격에 처치하나"의 분포를 동적 계획법으로 구하고(A.1.3), 두 분포를 결합해 1:1 승률을 공식으로 냅니다(A.4.1). 데미지 변동·특성이 있어 시뮬레이션으로 구한 승률에는 오차 범위를 붙이고(A.4.2), CP 순서가 실제 강함 순서와 맞는지 확인합니다(A.4.3). 마지막으로 밸런스 도구가 어느 요소를 고칠지 보여 줍니다. 특성·아이템을 하나씩 꺼 보는 효과 분해와 스탯별 민감도는 반대 가정·토네이도(A.4.4), 목표 승률에 맞추는 최소 변경은 이분 탐색·공통 난수(A.1.4)입니다.

계산 순서	방법	절
① 상대를 몇 번째 공격에 처치하나	동적 계획법	A.1.3
② 1:1 승률 (변동·특성 없음)	독립 확률변수 결합	A.4.1
③ 시뮬레이션 승률의 오차 범위	이항분포·Wilson 구간	A.4.2
④ CP가 실제 강함 순서와 맞나	스피어만 순위 상관	A.4.3
⑤ 무엇이 승패를 좌우하나, 무엇부터 고칠까	반대 가정·토네이도 민감도	A.4.4
⑥ 목표 승률에 맞추는 최소 변경	이분 탐색·공통 난수	A.1.4
⑦ 실제로 싸우면 선공·남은 체력·오버킬은 (배틀 시뮬레이터)	몬테카를로 시뮬레이션	A.1.1

A.4.1 독립 확률변수로 1:1 승률 계산

기획 관점으로 풀어 보기

풀려는 문제

승률 행렬의 모든 칸을 시뮬레이션으로 채우면 칸마다 수천 번 전투가 필요하고 오차가 남습니다. 데미지 변동·특성이 없는 대결은 공식으로 정확히 풀 수 있습니다.

핵심 아이디어 — 각자 몇 대 만에 잡나

A가 B를 쓰러뜨리는 데 필요한 공격 수 N_A는 A의 공격·치명·명중과 B의 방어·체력으로만 정해지고, B가 A를 몇 대 만에 잡는지와는 상관없습니다(독립). 그래서 둘을 따로 구한 뒤, **"A가 n번째 공격으로 잡는 시각 전에 B가 아직 못 잡았을 확률"**을 모든 n에 대해 더하면 승률입니다. N_A의 분포는 A.1.3의 동적 계획법으로 구합니다.

예

```
A: 1초에 1회 (0, 1, 2, 3초 …). 3대(50%) 또는 4대(50%, 치명 여부)에 B 처치
B: 1초에 0.8회 (0, 1.25, 2.5초 …). 항상 3대에 A 처치

A가 3대(2초)에 잡는 경우: 그 전 B의 공격은 2번 → B는 아직 못 잡음 → A 승
A가 4대(3초)에 잡는 경우: 2.5초에 B의 3번째 공격 → B 승
P(A 승) = 0.5 × 1 + 0.5 × 0 = 50%
```

"공격 속도가 빠른 A가 치명이 터지느냐 하나로 승패가 갈린다"는 사실이 계산 과정에 그대로 보입니다. 이런 분해는 "왜 이 매치업이 50%인가"를 설명할 때 유용합니다.

원리와 공식

아이디어: A가 B를 쓰러뜨리는 데 필요한 공격 횟수 N_A와, B가 A를 쓰러뜨리는 데 필요한 횟수 N_B는 데미지 변동·특성이 없으면 서로 영향을 주지 않는(독립) 확률변수입니다. 그래서 "A가 n번째 공격으로 쓰러뜨리고, 그 전에 B가 못 쓰러뜨릴 확률"을 곱해 모든 n에 대해 더하면 승률이 됩니다.

```
시간 기반:  P(A 승) = Σₙ P(N_A = n) · P(N_B > m_B(n))
            m_B(n) = A의 n번째 공격 시각 전에 B가 하는 공격 수,  n번째 공격 시각 = (n − 1) ÷ 공격 속도
           (같은 시각이면 "동시 결정타" 설정: 반반 또는 무승부)
```

```
턴 교대:    P(A 승) = Σₙ P(N_A = n) · [ 선공 · S_B(n − 1) + (1 − 선공) · S_B(n) ],  S_B(m) = P(N_B > m)
무승부:     최대 시간·턴 안에 둘 다 못 쓰러뜨릴 확률
```

검증: Kalivra v1.0.1 엔진을 그대로 돌린 결과(턴 교대, 40만~140만 회)와 비교해 세 경우 모두 3σ 안에서 일치합니다 — K1 0.8(정확히), K2 0.72643 대 0.72676, K3 0.02640 대 0.02659. 시간 기반 공식은 무작위 설정 12개 × 두 방식에서 자체 시뮬레이션과 일치합니다.

워크벤치에서 쓰인 곳: 스탯·전투의 승률 행렬, 1:1 상세, 레벨별 승률, CP 대 승률, 밸런스 도구 (변동·특성이 없는 대결).

A.4.2 이항분포와 Wilson 점수 구간

기획 관점으로 풀어 보기

풀려는 문제

시뮬레이션 승률 52%가 진짜 우위인지 운인지 판단하려면 승률에 오차 범위가 필요합니다.

같은 52%라도 횟수에 따라 결론이 다릅니다

전적	95% 범위 (Wilson)	판단
52승 / 100전	42.3% ~ 61.5%	50% 포함 — 우위라 말할 수 없음
520승 / 1,000전	48.9% ~ 55.1%	아직 50% 포함
5,200승 / 10,000전	51.0% ~ 53.0%	50% 밖 — 우위로 판단

범위 폭은 횟수의 제곱근에 반비례하므로, 폭을 절반으로 줄이려면 4배를 더 돌려야 합니다. 범위가 50%를 포함하면 "아직 차이가 있다고 말할 수 없음"으로 읽으세요.

왜 Wilson 구간인가

흔한 공식 $\hat{p} \pm 1.96\cdot\sqrt{\hat{p}(1-\hat{p})/n}$은 승률이 0%·100%에 가까우면 망가집니다. 20전 20승이면 범위 폭이 0이 되어 "승률 100%가 확실하다"고 말하는 셈입니다. Wilson 구간은 같은 경우 **83.9% ~ 100%**로, "20번으로는 승률이 84% 이상이라는 것까지만 안다"고 정직하게 답합니다. 밸런스 작업에서는 압도적 상성 같은 극단적 매치업을 자주 보므로 이 차이가 중요합니다.

원리와 공식

이항분포: 성공 확률 p인 독립 시행 n번에서 성공 횟수의 분포. 시뮬레이션 승률 $\hat{p}$ = 이긴 횟수 ÷ n의 표준오차는 $\sqrt{p(1-p)/n}$.

Wilson 점수 구간: 정규근사 구간 $\hat{p} \pm 1.96\cdot\sqrt{\hat{p}(1-\hat{p})/n}$은 $\hat{p}$가 0이나 1에 가까울 때 0~1을 벗어나거나 너무 좁아집니다($\hat{p}$ = 0이면 폭 0). Wilson 구간은 "참값 p에서 $\hat{p}$가 나올 만한 범위"를 거꾸로 풀어 얻은 구간이라 항상 0~1 안에 있고 실제 포함 확률이 95%에 가깝습니다.

$$\text{중심} = (\hat{p} + z^2/2n) \div (1 + z^2/n)$$

$$\text{반폭} = z \cdot \sqrt{\hat{p}(1-\hat{p})/n + z^2/4n^2} \div (1 + z^2/n), \quad z = 1.96\ (95\%)$$

워크벤치에서 쓰인 곳: 스탯·전투 시뮬레이션 승률의 95% 범위, 승률 행렬 칸의 ±오차, 1:1 상세의 시뮬레이션 열, 승률 수렴 그래프의 띠.

A.4.3 스피어만 순위 상관

기획 관점으로 풀어 보기

풀려는 문제

전투력(CP)은 "높을수록 강하다"는 약속입니다. 그 약속이 실제 전투 결과와 맞는지 확인하려는데, CP 값의 크기(1만 vs 1.2만)가 아니라 **순서**가 맞는지가 핵심입니다.

방법 — 순위로 바꿔서 비교

캐릭터	CP 순위	실제 강함 순위	차이 d
전사	1	1	0
궁수	2	2	0
법사	3	4	−1
도적	4	3	+1

$$\rho = 1 - 6\cdot\Sigma d^2 \div (n(n^2-1)) = 1 - 6 \times 2 \div (4 \times 15) = 0.8$$

1이면 순서가 완벽히 같고, 0이면 무관, −1이면 정반대입니다.

읽는 법

캐릭터가 4명이면 한 쌍만 뒤집혀도 0.8로 떨어지고, 40명이면 한 쌍이 뒤집혀도 0.9998입니다. 값

> 자체보다 "CP가 높은데 지는 쌍" 목록을 함께 보세요. 그 쌍이 CP 공식이 놓치는 요소(공격 속도, 치명, 상성)를 가리킵니다.

원리와 공식

무엇인가: 두 값의 **순위**가 얼마나 같은 순서인지 재는 상관계수입니다. 값을 순위(1등, 2등 …)로 바꾼 뒤 순위끼리의 피어슨 상관을 구합니다. 값의 크기 차이가 아니라 순서만 보므로, "CP 순서가 실제 강함 순서와 맞나"를 묻기에 알맞습니다.

```
ρ = 순위끼리의 피어슨 상관 = Σ(rx − r̄x)(ry − r̄y) ÷ √( Σ(rx − r̄x)² · Σ(ry − r̄y)² )
동점은 평균 순위 (2등·3등 동점이면 둘 다 2.5등)
동점이 없으면 ρ = 1 − 6 Σ d² ÷ ( n(n² − 1) )
```

읽는 법: 1 = 순서 완전히 같음, 0 = 관계 없음, −1 = 정반대. 엔티티가 4개면 한 쌍만 뒤바뀌어도 0.8이 됩니다. 엔티티가 적을수록 한 쌍의 영향이 크므로, 값보다 "CP가 높은데 지는 쌍" 목록을 함께 보세요.

워크벤치에서 쓰인 곳: 스탯·전투 KPI "CP와 실제 강함의 순위 상관", 브리핑 지표.

A.4.4 반대 가정과 토네이도 민감도

기획 관점으로 풀어 보기

반대 가정 — 원인을 "바꿔 보고" 판정

"이 캐릭터가 이기는 게 특성 때문인가, 스탯 때문인가?"는 승률 숫자를 나란히 놓아서는 알기 어렵습니다. 특성과 스탯이 함께 작용하기 때문입니다. 반대 가정은 의심 요소를 실제로 꺼서(또는 바꿔서) 다시 계산하고, 결과가 얼마나 달라지는지로 그 요소의 몫을 판정합니다. 스탯·전투 탭의 아이템·특성 효과 분해가 이 방식입니다. 상관관계가 아니라 **"이것을 빼면 이만큼 달라진다"**는 답입니다.

토네이도 — 무엇부터 손댈지

변수를 하나씩 ±10% 바꾸고 나머지는 그대로 둔 채 결과 변화를 막대로 그립니다. 공통 예제(평균 7,500원)에 적용하면

변수 (±10%)	결과 범위	막대 길이
+2 성공률 (36% ~ 44%)	8,333 ~ 6,818원	1,515원

변수 (±10%)	결과 범위	막대 길이
시도 비용 (900 ~ 1,100원)	6,750 ~ 8,250원	1,500원
+1 성공률 (45% ~ 55%)	8,056 ~ 7,045원	1,010원

같은 10%라도 +2 성공률이 +1 성공률보다 1.5배 영향이 큽니다(A.2.2의 민감도와 같은 결론). 막대가 긴 변수부터 위에 쌓으면 회오리 모양이 되고, 맨 위가 가장 효율적인 조정 손잡이입니다. 스탯·전투 밸런스 도구의 민감도는 같은 방식으로 "어느 스탯을 바꾸면 승률이 가장 많이 움직이나"를 보여 줍니다.

한계

한 번에 하나씩만 바꾸므로 두 변수를 함께 바꿀 때의 상호작용은 보이지 않습니다. ±10%는 작은 변화라 크게 바꾸면 순서가 달라질 수 있습니다.

원리와 공식

반대 가정(counterfactual): "만약 이 부분이 달랐다면 결과가 어땠을까"를 실제로 바꿔 다시 계산해 보는 방법입니다. 상관관계나 숫자 비교만으로 원인을 추정하는 것보다 직접적입니다. 스탯·전투 탭은 아이템·특성을 하나씩 끄고 다시 계산해, 각 요소가 승률·처치 시간에 얼마나 기여하는지 보여 줍니다.

토네이도(일변수) 민감도: 변수를 하나씩 ±p% 바꾸고 나머지는 그대로 둔 채 결과 변화를 막대로 그립니다. 영향이 큰 변수부터 위에 쌓아 회오리 모양이 됩니다.

한계: 한 번에 하나만 바꾸므로 두 변수를 함께 바꿀 때의 상호작용은 보이지 않습니다. 또 ±p%는 국소적 변화라, 크게 바꾸면 순위가 달라질 수 있습니다.

워크벤치에서 쓰인 곳: 스탯·전투 밸런스 도구의 민감도, 아이템·특성 효과 분해(하나씩 끄기).